Лабораторная работа 7

Diffusion и Score-Based Generative Models

Noise schedule, оценка score, численный reverse process и computational trade-offs

Курс «Нейронные сети»

Редакция 4 от 8 октября 2026 г.

1 Введение

Лабораторная работа 7 продолжает генеративный блок курса и непосредственно опирается на ЛР6, где исследованы Energy-Based Models и Normalizing Flows. В ЛР6 мы работали с двумя подходами: EBM задаёт ненормированную плотность $p_\theta(x) \propto \exp(-E_\theta(x))$; Flow задаёт точную плотность через обратимое отображение $x \leftrightarrow z$ с вычислимым якобианом. В данной работе рассматривается третий подход: вместо явной плотности или энергии моделируется процесс постепенного разрушения данных гауссовским шумом и обучается обратный процесс восстановления [9, 10].

Связь с ЛР6: для энергетической модели score равен градиенту энергии со знаком минус, $\nabla_x \log p_\theta(x) = -\nabla_x E_\theta(x)$ (формула (6.8) ЛР6). Score-based модель аппроксимирует градиент log-плотности напрямую, $s_\theta(x, t) \approx \nabla_x \log p_t(x)$, причём сразу на семействе зашумлённых распределений p_t [13, 14].

Все эксперименты выполняются в проекте из ЛР0: у каждого запуска есть `EXP_ID`, `RUN_ID`, паспорт и итоговая конфигурация. Числа в отчёте сопровождаются ссылками на `RUN_ID`.

2 Цель работы

Цель работы — реализовать диффузионную модель для двумерных данных. Требуется экспериментально установить, как качество порождённых примеров зависит от расписания шума, точности сети и числа шагов сэмплера. Отдельная задача — научиться разделять ошибку сети и ошибку численной схемы.

Как качество порождённых примеров зависит от расписания шума, обученной сети и сэмплера? Какая часть ошибки принадлежит сети, а какая — численной схеме сэмплирования?

3 Что нужно знать заранее

Если какой-то пункт незнаком, восполните пробел по указанному источнику.

- Нормальное и условное распределения, математическое ожидание, смесь распределений и расстояние Кульбака–Лейблера — [2, гл. 3].
- Порождающие модели и вариационная оценка правдоподобия — [1, разд. 8.2, 10.3, 10.4].
- Шумоподавляющий автокодировщик и сопоставление score (score matching; в переводе [2] — «сопоставление рейтингов») — [2, разд. 14.5, 18.4, 18.5].
- Общая идея диффузионных моделей — курс Hugging Face [5]; разбор реализации DDPM [6].
- Распределения и градиенты в PyTorch — `torch.distributions` [3], `torch.autograd.grad` [4].
- Проект из ЛР0 и правило сравнения по трём seeds из ЛР1.

4 Результаты обучения

После выполнения работы вы сможете:

1. объяснять прямой процесс и выводить формулу зашумления за один шаг;
2. реализовывать линейное и косинусное расписания шума и проверять согласование конечного шума с масштабом данных;
3. обучать сеть с условием по времени предсказывать шум и объяснять, почему это оценка поля score;
4. отличать точное условное распределение шага назад от модели обратного перехода;
5. реализовывать сэмплеры DDPM и DDIM и сравнивать их по качеству и числу вызовов сети;
6. разделять ошибку сети и ошибку численной схемы;
7. измерять покрытие мод и делать выводы по трём seeds.

5 Модели и обозначения

5.1 Прямой процесс: от данных к шуму

Прямой процесс (forward process) постепенно превращает данные в шум. На каждом шаге точка немного сжимается к нулю, и к ней прибавляется гауссовский шум:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\big(x_t;\ \sqrt{1-\beta_t}\,x_{t-1},\ \beta_t I\big), \qquad t = 1, \dots, T. \tag{7.1}$$

Здесь $x_0 \in \mathbb{R}^d$ — объект данных, x_t — его версия после t шагов, $\beta_t \in (0,1)$ — дисперсия шума шага. $\mathcal{N}(x; m, S)$ — нормальная плотность со средним m и ковариацией S, I — единичная матрица. Набор β_t — расписание шума (noise schedule). Множитель $\sqrt{1-\beta_t}$ сохраняет масштаб: дисперсия остаётся равной 1.

Шаги не обязательно выполнять по одному. Обозначим $\alpha_t = 1 - \beta_t$ и $\bar{\alpha}_t = \alpha_1 \alpha_2 \cdots \alpha_t$; по соглашению $\bar{\alpha}_0 = 1$. Тогда зашумление до уровня t выполняется сразу:

$$q(x_t \mid x_0) = \mathcal{N}\big(x_t;\ \sqrt{\bar{\alpha}_t}\,x_0,\ (1-\bar{\alpha}_t)I\big), \qquad \text{то есть} \qquad x_t = \sqrt{\bar{\alpha}_t}\,x_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon, \tag{7.2}$$

где $\epsilon \sim \mathcal{N}(0, I)$. Это доказывается по индукции. Шаг (7.1) умножает коэффициент при x_0 на $\sqrt{\alpha_t}$. Дисперсии независимых шумов складываются: $\alpha_t(1 - \bar{\alpha}_{t-1}) + \beta_t = 1 - \bar{\alpha}_t$. Формула (7.2) — главная: x_t для обучения получается одним вычислением, без t шагов (7.1).

Сколько сигнала осталось, показывает отношение сигнал/шум (signal-to-noise ratio):

$$\mathrm{SNR}(t) = \frac{\bar{\alpha}_t}{1 - \bar{\alpha}_t}, \qquad \lambda(t) = \ln \mathrm{SNR}(t). \tag{7.3}$$

Оно строго убывает с ростом t, и расписания удобно сравнивать по графику $\lambda(t)$. В работе сравниваются два расписания [10, 11]:

$$\begin{aligned} &\text{линейное:} \quad \beta_t = \beta_{\min} + \frac{t-1}{T-1}\,(\beta_{\max} - \beta_{\min}); \\ &\text{косинусное:} \quad \bar{\alpha}_t = \frac{f(t)}{f(0)}, \quad f(t) = \cos^2\!\Big(\frac{t/T + \delta}{1+\delta} \cdot \frac{\pi}{2}\Big), \quad \beta_t = \min\Big(1 - \frac{\bar{\alpha}_t}{\bar{\alpha}_{t-1}},\ 0{,}999\Big), \end{aligned} \tag{7.4}$$

где $\delta = 0{,}008$ не даёт β_t стать слишком малым в начале. В косинусном расписании $f(T) = 0$, поэтому до ограничения $\bar{\alpha}_T = 0$. Тогда β_t ограничивают числом 0,999 и заново считают $\bar{\alpha}_t$ как произведение $(1 - \beta_s)$: так $\bar{\alpha}_T > 0$.

Линейное расписание с $T = 1000$ и β от 10^{-4} до 0,02 даёт $\bar{\alpha}_T \approx 4 \cdot 10^{-5}$. Около трети шагов имеет SNR $< 0{,}01$. У косинусного расписания таких шагов около 6%, а $\bar{\alpha}_T \approx 2{,}4 \cdot 10^{-9}$. Поскольку $\ln \bar{\alpha}_T \approx -\sum_t \beta_t$, при меньшем T верхнюю границу β увеличивают обратно пропорционально T. Нижняя граница мала и почти не меняет сумму. Иначе данные не стираются до конца.

Конечный шум и масштаб данных. Генерация начинается с $x_T \sim \mathcal{N}(0, I)$, а настоящее x_T для данного x_0 имеет среднее $\sqrt{\bar{\alpha}_T}\, x_0$. Расхождение между ними

$$D_{\mathrm{KL}}\big(q(x_T \mid x_0) \,\|\, \mathcal{N}(0, I)\big) = \tfrac{1}{2}\big[\bar{\alpha}_T \|x_0\|^2 - d\,\bar{\alpha}_T - d\ln(1 - \bar{\alpha}_T)\big] \approx \tfrac{1}{2}\,\bar{\alpha}_T\,\|x_0\|^2, \tag{7.5}$$

где $\|\cdot\|$ — евклидова норма; приближение верно при малом $\bar{\alpha}_T$. Расхождение растёт с квадратом масштаба данных. При $T = 500$ и β от 10^{-4} до 0,02 получается $\bar{\alpha}_T = 6{,}35 \cdot 10^{-3}$. Для $\|x_0\| = 20$ расхождение равно 1,27 нат. Для стандартизованных данных с $\|x_0\|^2 \approx d = 2$ оно примерно в 200 раз меньше. Поэтому данные стандартизуют, а расписание обязано давать $\bar{\alpha}_T \le \bar{\alpha}_{\max} = 10^{-3}$.

Подробнее: [10, 11]; курс Hugging Face [5]; параметры расписаний в библиотеке Diffusers [7].

5.2 Сеть и обучение предсказанием шума

Сеть учится по зашумлённой точке и номеру шага угадывать добавленный шум. Берутся x_0 из `train`, номер шага t, равномерно распределённый на $\{1, \dots, T\}$, и шум $\epsilon \sim \mathcal{N}(0, I)$. По (7.2) вычисляется x_t. Сеть $\epsilon_\theta(x_t, t)$ с параметрами θ минимизирует среднеквадратичную ошибку

$$\mathcal{L}_{\mathrm{simple}}(\theta) = \mathbb{E}_{t, x_0, \epsilon}\,\Big\|\epsilon - \epsilon_\theta\big(\sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\,\epsilon,\ t\big)\Big\|^2, \tag{7.6}$$

где $\mathbb{E}$ — математическое ожидание.

Из предсказанного шума $\hat{\epsilon} = \epsilon_\theta(x_t, t)$ получается оценка исходного объекта: (7.2) решается относительно x_0,

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t}\,\hat{\epsilon}}{\sqrt{\bar{\alpha}_t}}, \qquad \|x_0 - \hat{x}_0\|^2 = \frac{\|\epsilon - \hat{\epsilon}\|^2}{\mathrm{SNR}(t)}. \tag{7.7}$$

По второму равенству норма ошибки по x_0 больше нормы ошибки по шуму в $1/\sqrt{\mathrm{SNR}(t)}$ раз. При $\mathrm{SNR}(T) = 4 \cdot 10^{-5}$ множитель примерно равен 158.

Номер шага подаётся в сеть через синусоидальное вложение [20, 10]:

$$\gamma(t) = \big[\sin(\omega_j t),\ \cos(\omega_j t)\big]_{j=0}^{d_e/2-1}, \qquad \omega_j = 10000^{-2j/d_e}, \qquad e_t = \mathrm{MLP}\big(\gamma(t)\big), \tag{7.8}$$

где d_e — чётная размерность вложения, MLP — небольшой многослойный перцептрон. Вектор e_t подаётся в каждый скрытый слой сети, а не один раз на входе.

Подробнее: [10]; реализация DDPM в блоге Hugging Face [6]; [1, разд. 8.2].

5.3 Обратный процесс: точное распределение и модель

Для генерации нужен путь назад, от x_t к x_{t-1}. Здесь важно различать три распределения.

Точное условное распределение. Если исходный объект x_0 известен, шаг назад гауссовский и вычисляется точно [10]:

$$\begin{gathered} q(x_{t-1} \mid x_t, x_0) = \mathcal{N}\big(x_{t-1};\ \tilde{\mu}_t(x_t, x_0),\ \tilde{\beta}_t I\big), \\ \tilde{\mu}_t(x_t, x_0) = \frac{\sqrt{\bar{\alpha}_{t-1}}\,\beta_t}{1-\bar{\alpha}_t}\, x_0 + \frac{\sqrt{\alpha_t}\,(1-\bar{\alpha}_{t-1})}{1-\bar{\alpha}_t}\, x_t, \qquad \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\,\beta_t. \end{gathered} \tag{7.9}$$

При $\bar{\alpha}_0 = 1$ получаем $\tilde{\beta}_1 = 0$: последний шаг назад не содержит шума.

Истинный обратный переход. При генерации x_0 неизвестен. Шаг назад — это (7.9), усреднённое по всем x_0, которые могли породить x_t:

$$q(x_{t-1} \mid x_t) = \int q(x_{t-1} \mid x_t, x_0)\, q(x_0 \mid x_t)\, dx_0. \tag{7.10}$$

Это смесь гауссовских распределений с весами $q(x_0 \mid x_t)$. В общем случае, например для многомодальных данных, она не гауссовская.

Модель обратного перехода. Модель выбирает гауссовское распределение и подставляет в $\tilde{\mu}_t$ оценку $\hat{x}_0$ из (7.7):

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}\big(x_{t-1};\ \mu_\theta(x_t, t),\ \sigma_t^2 I\big), \qquad \mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}}\Big(x_t - \frac{\beta_t\, \epsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}}\Big), \tag{7.11}$$

где σ_t^2 — дисперсия шага, выбранная заранее: $\tilde{\beta}_t$ (основной вариант) или β_t [10]. Гауссовская модель — приближение к (7.10). Оно становится точным лишь в пределе малых β_t [17, 9], поэтому DDPM делает сотни маленьких шагов.

При $\sigma_t^2 = \tilde{\beta}_t$ расхождение D_{KL} между (7.9) и (7.11) пропорционально $\|\epsilon - \epsilon_\theta\|^2$. Сумма таких расхождений входит в вариационную оценку правдоподобия, а $\mathcal{L}_{\mathrm{simple}}$ получается из неё отбрасыванием весов шагов [10]. Поэтому $\mathcal{L}_{\mathrm{simple}}$ не оценивает правдоподобие и не сравнивается с NLL потока из ЛР6.

Подробнее: [10, 9]; вариационная оценка — [1, разд. 10.3]; [7].

5.4 Связь с полем score

Score плотности $p(x)$ — вектор $\nabla_x \ln p(x)$. Он указывает, где плотность растёт быстрее всего. Для $\mathcal{N}(0, 1)$ score равен $-x$: в точке $x = 2$ он равен -2 и направлен к центру [13].

Для зашумления (7.2) score условного распределения выражается через шум:

$$\nabla_{x_t} \ln q(x_t \mid x_0) = -\frac{x_t - \sqrt{\bar{\alpha}_t}\, x_0}{1 - \bar{\alpha}_t} = -\frac{\epsilon}{\sqrt{1 - \bar{\alpha}_t}}. \tag{7.12}$$

Распределение всех x_t на шаге t — смесь $p_t(x_t) = \int q(x_t \mid x_0)\, p_{\text{data}}(x_0)\, dx_0$, где p_{data} — распределение данных. Дифференцирование под интегралом показывает: score смеси — среднее условных score,

$$\nabla_{x_t} \ln p_t(x_t) = \mathbb{E}_{x_0 \sim q(x_0 \mid x_t)} \big[\nabla_{x_t} \ln q(x_t \mid x_0) \big] = -\frac{\mathbb{E}[\epsilon \mid x_t]}{\sqrt{1 - \bar{\alpha}_t}}, \tag{7.13}$$

где $\mathbb{E}[\epsilon \mid x_t]$ — среднее шума по всем способам получить данное x_t.

Ошибка (7.6) минимальна, когда сеть выдаёт условное среднее $\mathbb{E}[\epsilon \mid x_t]$. Поэтому обученная сеть задаёт оценку поля score:

$$s_\theta(x_t, t) = -\frac{\epsilon_\theta(x_t, t)}{\sqrt{1 - \bar{\alpha}_t}} \approx \nabla_{x_t} \ln p_t(x_t). \tag{7.14}$$

Такое обучение называют шумоподавляющим сопоставлением score (denoising score matching) [15, 13]. Предсказание шума и оценка score — одна задача в разных единицах [14].

Сеть приближает среднее $\mathbb{E}[\epsilon \mid x_t]$, а не сам шум. Поэтому ошибку предсказания шума нельзя сделать нулевой. При малом шуме оптимальный прогноз почти нулевой, и неустранимая ошибка близка к 1 на координату. С идеальной сетью оценка (7.7) равна $\mathbb{E}[x_0 \mid x_t]$ (формула Твиди [16]). При большом шуме это среднее по многим модам, точка между ними.

Точный score для смеси гауссиан. Пусть $p_{\text{data}} = \sum_m \pi_m \mathcal{N}(\mu_m, \Sigma_m)$ — смесь с весами π_m, центрами μ_m и ковариациями Σ_m. Зашумление (7.2) снова даёт смесь:

$$p_t(x) = \sum_m \pi_m \mathcal{N}\big(x;\ \sqrt{\bar{\alpha}_t}\, \mu_m,\ \bar{\alpha}_t \Sigma_m + (1 - \bar{\alpha}_t) I\big). \tag{7.15}$$

Точный score $s^\star(x, t) = \nabla_x \ln p_t(x)$ вычисляется автоматическим дифференцированием [3, 4]. При большом t компоненты сливаются в одну, близкую к $\mathcal{N}(0, I)$, и поле гладкое. При малом t поле направлено к ближайшей моде. Точный предсказатель шума $\epsilon^\star(x, t) = -\sqrt{1 - \bar{\alpha}_t}\, s^\star(x, t)$ — эталон для сети: его ошибка неустранима.

Подробнее: [2, разд. 14.5, 18.4, 18.5]; [13, 15].

5.5 Сэмплеры DDPM и DDIM

Сэмплер проходит путь от шума x_T к примеру x_0. Число вызовов сети на один пример обозначается NFE (number of function evaluations).

DDPM [10] начинает с $x_T \sim \mathcal{N}(0, I)$ и проходит все T шагов модели (7.11):

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t z_t, \qquad z_t \sim \mathcal{N}(0, I), \qquad t = T, T-1, \ldots, 1. \tag{7.16}$$

По умолчанию $\sigma_t^2 = \tilde{\beta}_t$; вариант $\sigma_t^2 = \beta_t$ задаётся отдельной конфигурацией. На последнем шаге шум не добавляется, и выход равен $\mu_\theta(x_1, 1)$. Отсечение $\hat{x}_0$ по диапазону данных по умолчанию выключено. Для DDPM NFE $= T$.

DDIM [12] переходит между произвольными моментами $0 = \tau_0 < \tau_1 < \cdots < \tau_K = T$. На шаге $i = K, \dots, 1$ сеть вызывается один раз, $\hat{\epsilon}_i = \epsilon_\theta(x_{\tau_i}, \tau_i)$:

$$\hat{x}_0 = \frac{x_{\tau_i} - \sqrt{1-\bar{\alpha}_{\tau_i}}\,\hat{\epsilon}_i}{\sqrt{\bar{\alpha}_{\tau_i}}}, \qquad x_{\tau_{i-1}} = \sqrt{\bar{\alpha}_{\tau_{i-1}}}\,\hat{x}_0 + \sqrt{1-\bar{\alpha}_{\tau_{i-1}}-\sigma_i^2}\,\hat{\epsilon}_i + \sigma_i z_i,$$
$$\sigma_i = \eta \sqrt{\frac{1-\bar{\alpha}_{\tau_{i-1}}}{1-\bar{\alpha}_{\tau_i}}}\sqrt{1-\frac{\bar{\alpha}_{\tau_i}}{\bar{\alpha}_{\tau_{i-1}}}}, \tag{7.17}$$

Здесь $\eta \in [0,1]$ — параметр случайности, $z_i \sim \mathcal{N}(0, I)$. По шуму оценивается $\hat{x}_0$, затем он зашумляется до уровня τ_{i-1} тем же $\hat{\epsilon}_i$. При $\eta = 0$ случайность есть только в x_T. При $\eta = 1$ и $\tau_i = i$ получается $\sigma_i^2 = \tilde{\beta}_{\tau_i}$, то есть DDPM (7.16). На последнем шаге выход равен $\hat{x}_0$. Распределения $q(x_t \mid x_0)$ те же, что при обучении, поэтому сеть не переобучают [12]. Для DDIM NFE $= K$. Моменты τ_i — округление iT/K; совпадения убирают.

В Diffusers [7, 8] шаг 0 — это наш $t = 1$. Отсечение $\hat{x}_0$ (`clip_sample`) там включено по умолчанию: при сверке его выключают.

Подробнее: [12]; [7, 8]; [5].

5.6 Метрики

Ошибки по шагам вычисляются на `val` с фиксированными реализациями шума, общими для всех сравниваемых моделей:

$$E_\epsilon(t) = \frac{1}{d}\,\mathbb{E}\,\|\epsilon - \epsilon_\theta(x_t, t)\|^2, \qquad E_{x_0}(t) = \frac{1}{d}\,\mathbb{E}\,\|x_0 - \hat{x}_0\|^2 = \frac{E_\epsilon(t)}{\mathrm{SNR}(t)}. \tag{7.18}$$

Равенство следует из (7.7). «Трудность» шага зависит от единиц: при малом шуме E_ϵ велика, а E_{x_0} близка к нулю.

Разложение ошибки. Для смеси гауссиан ошибка сети делится на две части:

$$E_\epsilon(t) = E_\epsilon^\star(t) + E_{\mathrm{net}}(t),$$
$$E_{\mathrm{net}}(t) = \frac{1}{d}\,\mathbb{E}\,\big\|\epsilon^\star(x_t,t) - \epsilon_\theta(x_t,t)\big\|^2 = \frac{1-\bar{\alpha}_t}{d}\,\mathbb{E}\,\big\|s^\star(x_t,t) - s_\theta(x_t,t)\big\|^2, \tag{7.19}$$

где $E_\epsilon^\star(t)$ — неустранимая ошибка точного предсказателя $\epsilon^\star$, $E_{\mathrm{net}}(t)$ — собственная ошибка сети. Разложение верно, так как $\epsilon^\star = \mathbb{E}[\epsilon \mid x_t]$ и остаток $\epsilon - \epsilon^\star$ не коррелирует ни с какой функцией от x_t.

Качество примеров. Сравниваются $N = 5000$ примеров модели и точки `test`, в исходных единицах данных:

$$\mathrm{Cov} = \frac{\#\{m < M :\ \hat{p}_m \ge p_m/2\}}{M}, \qquad D_{\mathrm{mode}} = \frac{1}{2}\sum_{m=0}^{M} |p_m - \hat{p}_m|,$$
$$\mathrm{MMD}^2 = \mathbb{E}\,k(x,x') + \mathbb{E}\,k(y,y') - 2\,\mathbb{E}\,k(x,y), \qquad k(x,y) = \exp\!\big(-\|x-y\|^2/2h^2\big). \tag{7.20}$$

M — число мод. `mode_id` берёт ближайший центр μ_m, если расстояние меньше 3ρ. Иначе точка попадает в категорию $m = M$. Доли p_m считают так же по 10^5 точкам цели. Вне шаров лежит около 1,1% массы: для двумерного гауссиана это $e^{-9/2}$. Соседние центры удалены примерно на 10ρ, шары не пересекаются. Мода представлена при $\hat{p}_m \ge p_m/2$. D_{mode} — расстояние полной вариации. В MMD^2 [21] точки x, x' берутся из цели, y, y' — примеры. Полоса h — медиана попарных расстояний в `train`, одна на всю работу.

Уровень шума метрики — её значение, когда вместо примеров модели взяты N свежих точек самой цели. Повторите расчёт 5 раз и запишите среднее и стандартное отклонение. Различия ниже этого уровня не интерпретируются.

Парные разности. Основное сравнение конфигураций A и B выполняется на трёх общих seeds $s = 1, 2, 3$, как в ЛР1:

$$\Delta_s = Q(\mathrm{A}, s) - Q(\mathrm{B}, s), \qquad \bar{\Delta} = \frac{1}{3}\sum_{s=1}^{3} \Delta_s, \qquad s_\Delta = \sqrt{\frac{1}{2}\sum_{s=1}^{3}(\Delta_s - \bar{\Delta})^2}, \tag{7.21}$$

где $Q(\cdot, s)$ — метрика конфигурации при seed s, $\bar{\Delta}$ — средняя разность, s_Δ — её стандартное отклонение.

Подробнее: [21]; [10].

5.7 Сводка обозначений

Обозначение	Смысл
x_0, x_t, T, β_t, $\bar{\alpha}_t$	данные, шаг, длина, дисперсия, произведение α
SNR, λ, ϵ_θ, $\hat{x}_0$	сигнал/шум, его логарифм, сеть, оценка исходного объекта
$\tilde{\mu}_t$, $\tilde{\beta}_t$, μ_θ	точный шаг назад и модель
s_θ, $s^\star$, $\epsilon^\star$	score сети, точный score и шум
τ_i, K, η, NFE	сетка DDIM, шаги, случайность, вызовы сети
E_ϵ, E_{x_0}, E_{net}	ошибки по шуму, по x_0 и ошибка сети
Cov, D_{mode}, MMD^2, Δ_s	покрытие, частоты, ядро, парные разности

6 Данные и оборудование

Цель — смесь M гауссовских распределений на плоскости ($d = 2$) с центрами $\mu_m = (\cos(2\pi m/M), \sin(2\pi m/M))$ и ковариациями $\rho^2 I$. Мода $m = 0$ редкая: её вес π_0, остальные моды делят оставшуюся вероятность поровну. Значения M, ρ и π_0 задаёт профиль.

Генератор даёт `train` (20 000 точек), `val` (5 000) и `test` (5 000), каждую со своим seed. Параметры и seeds входят в паспорт и служат отпечатком данных. Роль `cal` в паспорте пустая и до ЛР8 не читается. Данные стандартизуют по `train`. Модель и (7.15) работают в этих координатах, метрики — в исходных.

Оборудование. Видеокарта не нужна. Обучение двумерной модели занимает на процессоре минуты, полный проход DDPM для 5000 точек — до минуты. Если это слишком медленно, уменьшите ширину сети до 128. Видеокарта нужна только для дополнительного задания с изображениями.

7 Индивидуальный профиль

Параметры ЛР7 вычисляются функцией `lab_pick` из ЛР1 по хешу номера зачётки вместе с номером работы. Добавьте в генератор профиля три поля:

```
profile["lab7_T_beta"] = lab_pick(sid, 7, 0, 2,
    [[500, 1e-4, 0.04], [1000, 1e-4, 0.02]])      # T, beta_min, beta_max
profile["lab7_modes"] = lab_pick(sid, 7, 2, 4,
```

```
    [[6, 0.10], [8, 0.08], [10, 0.06]])            # M, rho
profile["lab7_rare_weight"] = lab_pick(sid, 7, 4, 6, [0.01, 0.02, 0.04])
```

Здесь `sid` — номер зачётки, `profile` — словарь из ЛР0. Оба варианта T и β дают $\bar{\alpha}_T \approx 4 \cdot 10^{-5}$. Второе поле — число мод M и отклонение ρ. Третье — вес π_0 редкой моды. Для номера `20250137` получаются $T = 500$, $\beta \in [10^{-4};\ 0{,}04]$, $M = 10$, $\rho = 0{,}06$, $\pi_0 = 0{,}04$.

8 Задания

8.1 Задание 1. Расписание шума и прямой процесс

Что сделать.

1. Реализуйте компонент расписания и зашумление за один шаг:

```
schedule = make_schedule(config)   # kind: linear | cosine; T; beta range
schedule.beta, schedule.alpha, schedule.alpha_bar   # length T+1, alpha_bar
    [0] = 1
schedule.snr, schedule.log_snr, schedule.posterior_var   # posterior_var =
    beta_tilde
x_t, noise = q_sample(x0, t, schedule, noise=None)
```

2. Реализуйте расписания (7.4). Для косинусного ограничьте $\beta_t \le 0{,}999$ и вычисляйте $\bar{\alpha}_t$ как произведение $(1 - \beta_s)$. Все величины считайте в `float64`.
3. Записывайте в итоговую конфигурацию вид расписания, T, диапазон β и $\bar{\alpha}_T$. При $\bar{\alpha}_T > \bar{\alpha}_{\max}$ запуск завершается с ошибкой.
4. Функция `q_sample` вычисляет x_t по (7.2). У каждого объекта batch свой t, коэффициенты транслируются на остальные оси (broadcasting). Аргумент `noise` подаёт заранее заданный шум для диагностики. В цикле обучения шаги (7.1) последовательно не применяются.
5. Реализуйте генератор цели и стандартизацию (раздел 6).

Что должно получиться. Все величины расписания вычисляются в одном компоненте, а x_t для любого t получается одним вызовом.

Как проверить себя. Тесты 1 и 2 проходят. Проверка прямого процесса в E1 укладывается в порог.

8.2 Задание 2. Сеть с условием по времени и её обучение

Что сделать.

1. Реализуйте сеть $\epsilon_\theta(x_t, t)$ — многослойный перцептрон ширины 256 и глубины 6. Вложение e_t строится по (7.8), $d_e = 128$. К выходу каждого скрытого слоя l прибавляется $W_l e_t$, где W_l — обучаемая матрица.
2. Реализуйте обучение по (7.6). По умолчанию: Adam, скорость обучения 10^{-3}, batch 512, $2 \cdot 10^4$ шагов. Сохраняйте контрольные точки и журнал $\mathcal{L}_{\text{simple}}$ на `train` и `val`.
3. Реализуйте интерфейс параметризации:

```
target  = param.training_target(x0, noise, t)   # noise in epsilon mode
x0_hat  = param.to_x0(model(x_t, t), x_t, t)
eps_hat = param.to_epsilon(model(x_t, t), x_t, t)
score   = -eps_hat / (1 - alpha_bar[t]).sqrt()
```

Основной режим — предсказание шума с преобразованием (7.7). Другие параметризации подключаются в дополнительном задании 1 без правки остального кода.

4. Реализуйте точный предсказатель $\epsilon^\star(x, t)$ с тем же интерфейсом, что у сети. Градиент он вычисляет внутри `torch.enable_grad()`, чтобы работать и в сэмплере.

Что должно получиться. Сеть и точный предсказатель взаимозаменяемы: любой код, принимающий сеть, принимает и $\epsilon^\star$.

Как проверить себя. На одних и тех же парах (x_0, ϵ) из `val` ошибка сети не ниже ошибки $\epsilon^\star$ с точностью до шума оценки. Иначе в коде ошибка: например, сеть видит сам шум. Тест 6 проходит.

8.3 Задание 3. Сэмплеры DDPM и DDIM

Что сделать.

1. Реализуйте DDPM по (7.16) с соглашениями раздела 5.5 и DDIM по (7.17) для любых τ и η. Выбранная τ сохраняется в артефактах запуска.

2. Оба сэмплера принимают функцию предсказания шума — сеть или $\epsilon^\star$ — и, по желанию, заранее заданные x_T и z:

```
x0, info = ddpm_sample(eps_fn, schedule, x_T, z=None, keep=None)
x0, info = ddim_sample(eps_fn, schedule, x_T, K, eta=0.0, keep=None)
# info: NFE, time, device, tau, saved intermediate states
```

3. NFE считается счётчиком вызовов `eps_fn`, время — с указанием устройства и числа примеров. Сэмплеры работают под `torch.no_grad()` и по запросу (`keep`) сохраняют промежуточные x_t.

Что должно получиться. Один сэмплер работает и с сетью, и с точным score; модели сравниваются при одинаковой случайности.

Как проверить себя. Тесты 4 и 5 проходят.

8.4 Задание 4. Диагностика по шагам и траекториям

Что сделать.

1. **Ошибки по шагам.** На сетке не менее чем из 20 значений t, включая 1 и T, вычислите E_ϵ, E_{x_0} (7.18), $E^\star_\epsilon$ и E_{net} (7.19). Постройте их на одном графике с логарифмической шкалой.

2. **Поле score.** На сетке 41×41 постройте поля s_θ и $s^\star$ не менее чем для четырёх уровней шума. Вычислите косинус угла между векторами и отношение их длин. Средние по сетке берите с весом $p_t(x)$ из (7.15): иначе их определяют пустые области, где сеть почти не обучалась.
3. **Траектории.** Запишите 1000 траекторий DDPM из фиксированных начальных шумов. На срезах $t/T \in \{0{,}8;\ 0{,}6;\ 0{,}4;\ 0{,}2;\ 0{,}05\}$ найдите долю траекторий, у которых ближайший из центров $\sqrt{\bar\alpha_t}\,\mu_m$ совпадает с итоговой модой.
4. **Качество примеров.** Реализуйте Cov, $\hat p_m$, D_{mode}, MMD^2 (7.20) и уровень шума метрик.

Что должно получиться. Для любой модели одной командой строятся кривые ошибок, поля score, траектории и сводка качества примеров.

Что добавляется в общий проект.

```
src/
|-- data/gmm2d.py          target, mode_id, exact score
|-- diffusion/             schedules.py  forward.py  objectives.py
|                          parameterizations.py  samplers.py
|-- models/diffusion/      time_embedding.py  mlp_eps.py
‘-- diagnostics/           timestep_error.py  score_field.py
                           trajectory.py  sample_quality.py
```

Сэмплеры и диагностика работают с любой сетью, предсказывающей шум.

Как проверить себя. Для точного предсказателя $E_{\text{net}}(t) = 0$, а косинус равен 1 на всей сетке. Тест 3 проходит.

9 Эксперименты

До запуска запишите гипотезу со ссылкой на формулу раздела 5. E1 и E2 — поисковые, хватает одного seed. Главный вывод проверяется в E3 на трёх seeds. Расхождение с гипотезой допустимо, если указана причина. Сравнивайте на одной цели, в одном масштабе и с общими начальными шумами.

9.1 Эксперимент E1. Расписания шума

Вопрос. Как линейное и косинусное расписания распределяют шум по шагам и как это сказывается на обучении и примерах?

Что сделать.

1. Для обоих расписаний с T из профиля постройте β_t, $\bar\alpha_t$ и $\lambda(t)$. Вычислите $\bar\alpha_T$ и среднее по `train` расхождение (7.5).
2. Проверьте прямой процесс. Для одной точки x_0 возьмите доли t/T из набора $\{0{,}1;\ 0{,}25;\ 0{,}5;\ 0{,}75;\ 1\}$ и получите $N = 10^4$ значений x_t функцией `q_sample`. По каждой координате сравните среднее $\hat m$ с $\sqrt{\bar\alpha_t}\,x_0$, а дисперсию $\hat v$ — с $1 - \bar\alpha_t$. Постройте облака x_t для всей `train`.

3. Обучите одну и ту же сеть с каждым расписанием при одинаковом числе шагов и одном seed. Сравните кривые $\mathcal{L}_{\text{simple}}$ и $E_\epsilon(t)$, а также MMD^2, Cov и D_{mode} для 5000 примеров DDPM. Постройте примеры поверх точек `test`.

Порог приёмки.

- $\bar{\alpha}_T \le \bar{\alpha}_{\max} = 10^{-3}$ для обоих расписаний; приведено значение (7.5) для стандартизованных данных.
- Для каждого t и каждой координаты $|\hat{m} - \sqrt{\bar{\alpha}_t}\, x_0| \le 4\sqrt{(1-\bar{\alpha}_t)/N}$ и $|\hat{v}/(1-\bar{\alpha}_t) - 1| \le 4\sqrt{2/N}$ — четыре стандартные ошибки среднего и дисперсии. При $N = 10^4$ вторая граница равна 0,057.
- Обе модели обучены одинаково, кроме расписания; все метрики приведены с `RUN_ID`.

Граница применимости. Уменьшите T в 10 раз, не меняя диапазон β линейного расписания. Вычислите $\bar{\alpha}_T$ и (7.5). Получите примеры DDPM с точным предсказателем $\epsilon^\star$: так влияние конечного шума видно без ошибки сети. Выполняется ли порог? Как изменились MMD^2 и D_{mode} и почему?

9.2 Эксперимент E2. Число шагов сэмплирования: DDPM и DDIM

Вопрос. Сколько шагов сэмплера нужно обученной модели? Чья ошибка растёт при сокращении шагов — сети или численной схемы?

Что сделать.

1. Возьмите модель E1 с линейным расписанием. Получите по 5000 примеров DDIM с $\eta = 0$ при $K \in \{5, 10, 20, 50, 100, T\}$ и DDPM при общем начальном шуме. Для каждого K запишите MMD^2, D_{mode}, Cov, NFE и время — медиану пяти повторов после прогревочного запуска.
2. Повторите измерения с точным предсказателем $\epsilon^\star$ вместо сети.
3. Повторите DDIM сети при $K = T$ с тремя начальными шумами и найдите стандартное отклонение s_{MMD} величины MMD^2.
4. Постройте MMD^2 и D_{mode} от NFE (логарифмическая шкала) для сети и $\epsilon^\star$ с отметкой уровня шума, а также время от NFE.

Разность кривых сети и $\epsilon^\star$ при одном K показывает вклад сети. Рост кривой $\epsilon^\star$ при малых K — вклад численной схемы.

Порог приёмки.

- NFE равно K для DDIM и T для DDPM. Время на один вызов сети при $K \ge 10$ отличается от своей медианы не более чем на 20%.
- С $\epsilon^\star$ при $K = T$ DDPM и DDIM дают MMD^2 не выше уровня шума плюс три его стандартных отклонения. Иначе ошибка в сэмплере: проверьте нумерацию шагов, $\bar{\alpha}_0 = 1$ и последний шаг.

- Указано наименьшее K, при котором MMD^2 сети отличается от значения при $K = T$ не более чем на $2s_{\mathrm{MMD}}$.

Граница применимости. Очень мало шагов: $K = 1$ и $K = 2$. При $K = 1$ DDIM возвращает $\hat{x}_0$, вычисленное прямо из чистого шума. Где окажутся примеры с $\epsilon^\star$? Сравните с формулой Твиди (раздел 5.4). Как ведут себя примеры сети и почему, согласно формуле (7.7)? С какого K примеры попадают в моды?

9.3 Эксперимент E3. Покрытие мод

Вопрос. Воспроизводит ли модель все моды цели вместе с их частотами? Выдерживает ли главный вывод работы смену seed?

Что сделать.

1. По результатам E1–E2 выберите конфигурации A и B, различающиеся ровно одним фактором. Например: косинусное и линейное расписания при одинаковом числе шагов обучения; DDIM с малым K и DDPM на одной модели.
2. До запусков запишите в `hypothesis.md` и зафиксируйте в Git: чем различаются A и B; что зафиксировано (цель, T, расписание, σ_t^2, сэмплер, K, η, число шагов обучения, seeds); ожидаемый знак разности D_{mode} и механизм со ссылкой на формулу; какой результат опровергнет гипотезу.
3. Выполните A и B с тремя общими seeds $s = 1, 2, 3$. Seed задаёт инициализацию сети, порядок batch, t и ϵ при обучении, начальный шум и z_t при сэмплировании.
4. Для каждого запуска получите 5000 примеров и вычислите Cov, частоты $\hat{p}_m$, D_{mode} и парные разности (7.21) для $Q = D_{\mathrm{mode}}$. Постройте диаграммы $\hat{p}_m$ рядом с p_m и все три Δ_s на одном графике.

Порог приёмки. Вывод «A и B различаются по D_{mode}» считается установленным, если все три Δ_s одного знака и $|\bar{\Delta}| \geq 2s_\Delta$. Иначе вывод формулируется как «различие не установлено». В отчёте приводятся все три Δ_s, уровень шума D_{mode}, а также Cov и частота редкой моды $\hat{p}_0$ каждого запуска. Расхождение с гипотезой не снижает оценку, если оно объяснено.

Граница применимости. Уменьшите вес редкой моды в 2 и в 4 раза (один seed, конфигурация A). При каком весе мода перестаёт быть представленной? Сколько точек `train` приходится на неё в этот момент? Три seeds описывают разброс на одной цели и не переносят вывод на другие цели — укажите это в отчёте.

10 Контрфактические вопросы

Ответ опирается на ваши числа со ссылками на `RUN_ID`. Если для ответа нужен дополнительный запуск, выполните его.

1. Увеличьте T в 10 раз при том же диапазоне β. Какими станут $\bar{\alpha}_T$ и расхождение (7.5)? Обязано ли качество примеров улучшиться? Во сколько раз вырастут NFE и время DDPM? Проверьте одним обучением.

2. Положите в DDPM $\epsilon_\theta \equiv 0$. Тогда каждый шаг делит x_t на $\sqrt{\alpha_t}$ и добавляет шум с дисперсией σ_t^2. Выведите и вычислите для своего расписания дисперсию x_0. Совпадает ли она с измеренной? Что это говорит о роли сети?

3. В E1 значения $\mathcal{L}_{\text{simple}}$ у двух расписаний различаются. Порождает ли модель с меньшей ошибкой лучшие примеры? Сопоставьте $\mathcal{L}_{\text{simple}}$ с MMD^2 и D_{mode} и объясните по кривым $E_\epsilon(t)$, учитывая долю шагов с $\text{SNR} < 0{,}01$.

4. Умножьте $E_{\text{net}}(T)$ из задания 4 на $1/\,\text{SNR}(T)$ для двух своих расписаний. По (7.7) это средний квадрат отклонения $\hat{x}_0$ сети от $\mathbb{E}[x_0 \mid x_T]$ на первом шаге DDIM. Видно ли это усиление ошибки в примерах DDIM при $K = 5$?

5. Найдите в E3 запуск с $\text{Cov} = 1$, но наибольшим D_{mode}. Какая мода недо- или перепредставлена? По траекториям из задания 4 определите, на каком уровне шума формируется её доля. Достаточно ли Cov для вывода о качестве модели?

11 Обязательные тесты

1. Расписание: $0 < \beta_t < 1$; $\bar{\alpha}_t$ и $\lambda(t)$ строго убывают; $\bar{\alpha}_0 = 1$; $\tilde{\beta}_1 = 0$; $\bar{\alpha}_T \le \bar{\alpha}_{\max}$.

2. `q_sample`: моменты x_t согласуются с (7.2) в пределах четырёх стандартных ошибок; коэффициенты верно транслируются на формы $(B, 2)$ и (B, C, H, W).

3. В `float64`: формула (7.7) с истинным шумом восстанавливает x_0 с точностью 10^{-8} при всех t; тождество $E_{x_0} = E_\epsilon/\,\text{SNR}$ выполняется; точный score совпадает с конечными разностями $\ln p_t$ с относительной точностью 10^{-4}.

4. Сэмплеры: в `float64` при одинаковых x_T и z DDIM с $\eta = 1$ и полной τ совпадает с DDPM с точностью 10^{-6}; при $\eta = 0$ один x_T даёт один пример; последний шаг возвращает $\hat{x}_0$ без шума; NFE равно числу вызовов сети; параметры сети не меняются, граф вычислений не строится.

5. Последовательность τ строго возрастает, $\tau_K = T$, все элементы лежат в $[1, T]$.

6. Вложение времени детерминировано, и $\gamma(t) \neq \gamma(t')$ при $t \neq t'$.

12 Что сдать

Сдаются модули заданий 1–4, конфигурации E1–E3, файл `hypothesis.md`, тесты и отчёт. Отчёт состоит из шести частей:

1. **Профиль и протокол:** T, β, M, ρ, π_0, сеть, число шагов обучения, масштаб стандартизации, полоса h.

2. **E1:** графики расписаний, $\bar{\alpha}_T$ и (7.5), проверка прямого процесса, сравнение моделей, граница применимости.

3. **E2:** кривые MMD^2 и D_{mode} от NFE для сети и $\epsilon^\star$, время, найденное K, примеры при $K = 1$ и 2.

4. **Диагностика:** E_ϵ, E_{x_0}, E_{net}; поля score с картами косинуса; доля траекторий, выбравших моду.

5. **E3:** гипотеза, частоты мод, Cov, все Δ_s, уровень шума, вывод.

6. **Ответы на контрфактические вопросы.**

Каждое число сопровождается ссылкой на `RUN_ID`. Графики строятся автоматически из сохранённых результатов. В подписи графика с примерами указаны расписание, сэмплер, K и η.

13 Критерии завершения и оценивания

Работа завершена, если одновременно:

- эксперименты E1–E3 выполнены, у каждого результата есть `RUN_ID`;
- тесты проходят;
- файл `hypothesis.md` зафиксирован в Git до запусков E3;
- расписание, $\bar{\alpha}_T$, σ_t^2, τ и η записаны в итоговой конфигурации каждого запуска;
- на все контрфактические вопросы даны ответы с собственными числами.

Веса частей оценки те же, что в ЛР0: реализация 0,15, экспериментальный протокол 0,30, диагностика 0,30, перенос 0,25.

14 Дополнительные задания

1. **Параметризации.** Обучите сеть, которая предсказывает x_0, и, по желанию, сеть для $v_t = \sqrt{\bar{\alpha}_t}\,\epsilon - \sqrt{1-\bar{\alpha}_t}\,x_0$ [18]. Архитектура, расписание и число шагов те же. Переведите предсказания в $\hat{\epsilon}$ и сравните кривые $E_\epsilon(t)$. По (7.7) ошибка по x_0 — это ошибка по шуму с весом $1/\operatorname{SNR}(t)$, а ошибка по v — с весом $1/\bar{\alpha}_t$. Как эти веса перераспределяют точность сети по шагам и как меняются MMD^2 и D_{mode}?

2. **Изображения и управление без классификатора.** Нужна видеокарта. Английское имя метода — classifier-free guidance. Обучите компактный U-Net [22] на MNIST [23] или FashionMNIST [24], пиксели в $[-1, 1]$. Сравните DDPM и DDIM по NFE и времени. Затем обучите $\epsilon_\theta(x_t, t, y)$: метка y с вероятностью 0,1 заменяется пустым условием $\varnothing$. При генерации [19]

$$\hat{\epsilon}_w = \epsilon_\theta(x_t, t, \varnothing) + w\,\big[\epsilon_\theta(x_t, t, y) - \epsilon_\theta(x_t, t, \varnothing)\big]. \tag{7.22}$$

   Здесь $w = 0$ — безусловная модель, $w = 1$ — условная. В [19] шкала сдвинута на единицу. При $w \in \{0, 1, 2, 4\}$ измерьте долю нужного класса и разнообразие внутри класса. Рост w сужает распределение: при точных score оно пропорционально $p_t(x \mid y)\,p_t(y \mid x)^{w-1}$. Тот же опыт можно поставить на двумерной цели, где условие — номер моды.

3. **Сравнение с моделями ЛР6.** Если есть обученные в ЛР6 энергетическая модель и поток, сравните их с диффузией на одной цели и в одном масштабе: Cov, D_{mode}, MMD^2 и стоимость примера. У диффузии это K вызовов сети, у энергетической модели — шаги Ланжевена, у потока — один проход. $\mathcal{L}_{\mathrm{simple}}$ с NLL потока не сравнивают (раздел 5.3). Какие ограничения ЛР6 снимает диффузия и чем платит?

15 Контрольные вопросы

1. Что делает один шаг прямого процесса и почему x_t можно получить из x_0 сразу?

2. Что показывает $\mathrm{SNR}(t)$ и чем линейное расписание отличается от косинусного?
3. Почему конечный шум должен быть согласован с масштабом данных?
4. Почему $q(x_{t-1} \mid x_t, x_0)$ гауссовское, а $q(x_{t-1} \mid x_t)$ в общем случае нет? Когда гауссовская модель шага назад точна?
5. Как предсказание шума связано с полем score?
6. Почему ошибку предсказания шума нельзя сделать нулевой и при каком шуме неустранимая ошибка наибольшая?
7. Чем DDIM отличается от DDPM и почему для него не нужно переобучать сеть?
8. Почему ошибка сети и ошибка сэмплера — разные источники и как их разделить в этой работе?

16 Итог

После ЛР7 диффузионная модель — это прямой процесс, сеть и сэмплер. Конечный шум согласован с данными, а сеть оценивает поле score. Качество примеров зависит от всех трёх частей. Точный score смеси отделяет ошибку сети от ошибки схемы.

Литература

[1] Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с.

[2] Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А. А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с.

[3] PyTorch Documentation: torch.distributions. https://docs.pytorch.org/docs/stable/distributions.html

[4] PyTorch Documentation: torch.autograd.grad. https://docs.pytorch.org/docs/stable/generated/torch.autograd.grad.html

[5] Hugging Face Diffusion Models Course: Unit 1. An Introduction to Diffusion Models. https://huggingface.co/learn/diffusion-course/unit1/1

[6] N. Rogge, K. Rasul. The Annotated Diffusion Model. Hugging Face Blog, 2022. https://huggingface.co/blog/annotated-diffusion

[7] Hugging Face Diffusers Documentation: DDPMScheduler. https://huggingface.co/docs/diffusers/api/schedulers/ddpm

[8] Hugging Face Diffusers Documentation: DDIMScheduler. https://huggingface.co/docs/diffusers/api/schedulers/ddim

[9] J. Sohl-Dickstein, E. Weiss, N. Maheswaranathan, S. Ganguli. Deep Unsupervised Learning using Nonequilibrium Thermodynamics. *ICML*, 2015.

[10] J. Ho, A. Jain, P. Abbeel. Denoising Diffusion Probabilistic Models. *NeurIPS*, 2020.

[11] A. Nichol, P. Dhariwal. Improved Denoising Diffusion Probabilistic Models. *ICML*, 2021.

[12] J. Song, C. Meng, S. Ermon. Denoising Diffusion Implicit Models. *ICLR*, 2021.

[13] Y. Song, S. Ermon. Generative Modeling by Estimating Gradients of the Data Distribution. *NeurIPS*, 2019.

[14] Y. Song, J. Sohl-Dickstein, D. P. Kingma, A. Kumar, S. Ermon, B. Poole. Score-Based Generative Modeling through Stochastic Differential Equations. *ICLR*, 2021.

[15] P. Vincent. A Connection Between Score Matching and Denoising Autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[16] B. Efron. Tweedie's Formula and Selection Bias. *Journal of the American Statistical Association*, 106(496):1602–1614, 2011.

[17] W. Feller. On the Theory of Stochastic Processes, with Particular Reference to Applications. *Proceedings of the Berkeley Symposium on Mathematical Statistics and Probability*, 403–432, 1949.

[18] T. Salimans, J. Ho. Progressive Distillation for Fast Sampling of Diffusion Models. *ICLR*, 2022.

[19] J. Ho, T. Salimans. Classifier-Free Diffusion Guidance. arXiv:2207.12598, 2022.

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, I. Polosukhin. Attention Is All You Need. *NeurIPS*, 2017.

[21] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, A. Smola. A Kernel Two-Sample Test. *JMLR*, 13:723–773, 2012.

[22] O. Ronneberger, P. Fischer, T. Brox. U-Net: Convolutional Networks for Biomedical Image Segmentation. *MICCAI*, 2015.

[23] Y. LeCun, L. Bottou, Y. Bengio, P. Haffner. Gradient-Based Learning Applied to Document Recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[24] H. Xiao, K. Rasul, R. Vollgraf. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747, 2017.